

Privacy-Focused Secure Notes Application

Abstract

This project presents the design and implementation of a privacy-focused, client-side encrypted notes application built using modern web technologies. The system ensures that all user-generated content remains confidential through cryptographic mechanisms executed entirely within the browser. It follows a zero-knowledge architecture, meaning no plaintext data is stored or transmitted outside the user's device. Encryption is implemented using AES-GCM for confidentiality and integrity, while PBKDF2 with SHA-256 derives a secure key from the user's password. A separate 256-bit vault key is generated to encrypt notes and is itself encrypted using the password-derived key. The application supports full CRUD operations, inactivity-based auto-lock, and secure logout, demonstrating how a secure, offline-first system can be developed without backend infrastructure.

Introduction

With the rapid growth of digital tools and cloud-based services, concerns about data privacy and unauthorized access have increased significantly. Many traditional note-taking applications rely on centralized servers for storage, exposing user data to risks such as breaches, insider threats, and unauthorized surveillance. This creates a strong need for systems that prioritize user privacy and data ownership.

The Secure Notes Application was developed to address these concerns by implementing a fully client-side encrypted note-taking system. All encryption and decryption operations occur within the browser, and no plaintext data is stored or transmitted externally. Even if local storage is accessed, the stored information remains encrypted and unusable without the correct credentials.

A key architectural decision is the separation between authentication and data encryption through the use of a randomly generated 256-bit vault key. The user's password is processed using PBKDF2 to derive a secure key that encrypts the vault key, rather than encrypting notes directly. Combined with inactivity-based auto-lock and session clearing mechanisms, this design ensures strong security while maintaining usability.

Tools Used

The project was implemented using the following technologies:

- **React** – Used to build a dynamic and responsive user interface.
- **TypeScript** – Provides strong typing, improving maintainability and reducing runtime errors.
- **Web Crypto API** – Implements AES-GCM encryption and PBKDF2 key derivation using browser-native cryptographic primitives.

- **IndexedDB (via idb library)** – Stores encrypted notes locally in the browser for offline-first functionality.
- **Tailwind CSS** – Used for modern, responsive UI styling.
- **Lucide React** – Provides user interface icons such as edit and delete buttons.
- **Vite** – Serves as the development build tool for optimized frontend performance.

These tools were selected to ensure a balance between security, performance, maintainability, and user experience.

Steps Involved in Building the Project

The development of the Secure Notes Application followed a structured and security-first approach:

1. Designed the system architecture with clear separation between password authentication and vault encryption mechanisms.
2. Implemented PBKDF2 with SHA-256 and 100,000 iterations to derive a secure encryption key from the user's password.
3. Generated a cryptographically secure 256-bit vault key using secure random number generation.
4. Encrypted the vault key using AES-GCM with the password-derived key.
5. Stored the encrypted vault key and salt in localStorage, ensuring that plaintext keys never persist.
6. Implemented encrypted note storage using IndexedDB, ensuring notes are stored only in encrypted form.
7. Developed full CRUD functionality including note creation, viewing, editing, and deletion.
8. Integrated confirmation dialogs for delete operations to prevent accidental data loss.
9. Added recovery key functionality to enable password reset without compromising encrypted notes.
10. Implemented auto-lock after inactivity and enforced logout on page refresh to enhance session security.
11. Ensured that encryption keys remain only in memory and are cleared upon logout.

Each stage of development prioritized security, usability, and modular code structure.

Conclusion

The Secure Notes Application demonstrates the effective integration of cybersecurity principles into modern frontend development by using AES-GCM for authenticated encryption and PBKDF2 for secure key derivation to ensure confidentiality and integrity of user data. The separation between password-derived keys and the vault key strengthens the system architecture and enables secure recovery functionality. By following a zero-knowledge, offline-first model and incorporating features such as encrypted storage, auto-lock, and structured CRUD operations, the project showcases a secure and practical note management solution built without backend infrastructure.